# Sprint 6 · Reto «Datos sucios, cliente impaciente»

**Esta es la ficha completa del reto, explicada paso a paso** para quien nunca ha trabajado con esta metodología ni con IA. Léela entera **antes de empezar** y vuelve a ella cuando tengas dudas.

- Si es tu primer reto, lee antes Cómo se trabaja un reto.
- Volver al README del sprint · Guía del sprint en PDF · versión ecofriendly para imprimir.
- **Este reto en PDF:** a color · ecofriendly, para imprimir.

## RETO Ficha del reto

### En pocas palabras (léelo primero)

**Qué os pide el cliente.** TurisData os envía por correo un fichero de reservas exportado **a mano desde varios sistemas** (`reservas_sucio.csv`). El gerente quiere un primer modelo de cancelaciones «para ayer», pero **no se fía de los datos** (con razón). Os pide dos cosas:

1. Un **informe corto** que diga **qué está mal en los datos y por qué**.
2. Un **proceso automático** que deje los datos listos cada vez que le llegue un fichero así, con un primer modelo sencillo de referencia (un *baseline*).

**Los datos reales están siempre sucios.** Fechas escritas de varias maneras, países con distintos nombres, números negativos imposibles, filas repetidas, huecos… Limpiarlos es entre el 60 y el 80 % del trabajo en un proyecto de IA. En este reto aprenderéis a **encontrar los defectos, decidir qué hacer con cada uno y dejarlo todo escrito en código** para que se pueda repetir.

**Qué vais a hacer, en seis pasos:**

1. **Hacer un inventario** de los defectos del fichero (qué, dónde, cuántas filas).
2. **Explicar la causa probable** de cada uno y **decidir qué tratamiento** le dais (y por qué).
3. **Escribir una función de limpieza** en código (`limpiar_reservas`). Nada de arreglar el CSV a mano.
4. **Montar un pipeline** de preprocesado (imputar huecos, escalar números, codificar categorías) que aprenda **solo con los datos de entrenamiento**.
5. **Entrenar un modelo de referencia (*baseline*)** y compararlo con un modelo «tonto» mediante validación cruzada.
6. **Escribir el informe y el README.**

**Qué vais a entregar:** un repositorio o carpeta con `informe_calidad.md`, `reto_S06.ipynb` y `README.md`.

**Qué vais a aprender haciéndolo:** a **desconfiar de los datos**, a limpiar de forma **reproducible** y a evitar la **fuga de información** del test al entrenamiento.

**Cuánto tiempo:** unas 2 horas y 5 minutos en equipo entre el planning, el reto y la entrega (4 y 5/11).

**Núcleo y extra.** Núcleo: inventario de defectos, función de limpieza y *baseline* con su *pipeline*. Extra: informe más detallado y comparativa de variantes de limpieza.

### Palabras que van a aparecer en este reto

| Palabra | Qué significa, en sencillo |
|---|---|
| **EDA** (análisis exploratorio) | Mirar los datos con tablas y gráficos antes de modelar, para entenderlos |
| **Defecto de calidad** | Un problema del dato: huecos (nulos), duplicados, valores imposibles, formatos mezclados… |
| **Imputar** | Rellenar un hueco con un valor razonable (la mediana, la categoría más frecuente…) |
| **Outlier (valor atípico)** | Un valor muy raro. A veces es un error; a veces es real. Hay que decidir |
| **Pipeline** | Una cadena de pasos de preparación y modelo que se ejecuta siempre igual |
| **Train / test** | Los datos con los que entrena el modelo (train) y los que quedan guardados para evaluarlo (test) |
| **Fuga de datos** | Cuando información del test «se cuela» en el entrenamiento y los resultados parecen mejores de lo que son |
| **Baseline** | Modelo sencillo de referencia. Cualquier modelo mejor debe superarlo |
| **DummyClassifier** | Un «modelo tonto» que, por ejemplo, siempre responde la clase más frecuente. Es el suelo con el que comparar |

| Palabra | Qué significa, en sencillo |
|---|---|
| **Validación cruzada** | Entrenar y evaluar varias veces con particiones distintas para tener una medida más fiable |
| **Reproducible** | Que otra persona pueda ejecutar el cuaderno de arriba abajo y obtener lo mismo |

**Contexto del cliente.** TurisData Canarias nos manda por correo un fichero exportado a mano desde varios sistemas: `reservas_sucio.csv`. Quieren un primer modelo de cancelaciones "para ayer", pero el gerente no se fía de los datos. Nos pide **un informe corto que le diga qué está mal y por qué**, y **un proceso que deje los datos listos cada vez que le llegue un fichero así**, con un modelo de referencia (*baseline*).

**Qué se entrega** (un repositorio o carpeta con):

1. `informe_calidad.md` (o PDF), 1–2 páginas: tabla de defectos (qué, dónde, cuántas filas, causa probable, tratamiento y su justificación) y las preguntas que haríais al cliente. Plantilla en `plantillas/informe_calidad_plantilla.md`.
2. `reto_S06.ipynb`: notebook con el EDA, la función de limpieza y el **pipeline** de preprocesado + un modelo baseline evaluado con validación cruzada.
3. `README.md` con instrucciones para reproducirlo (versiones de librerías, cómo ejecutar).

**Restricciones.**

- Solo librerías del curso (pandas, scikit-learn, matplotlib, seaborn…).
- Nada de editar el CSV a mano: toda corrección debe estar en código.
- Todo estadístico aprendido de los datos (medianas, medias, categorías) debe ajustarse **solo con train**.
- No se descarta ninguna fila sin justificarlo en el informe.

**Criterios de evaluación** (0–4 cada uno; misma rúbrica de todos los retos):

| Criterio | 1 · Insuficiente | 2 · Suficiente | 3 · Bueno | 4 · Excelente |
|---|---|---|---|---|
| Corrección técnica | Errores que invalidan el resultado | Correcto con fallos menores | Correcto y bien justificado | Correcto, justificado y con alternativas comparadas |
| Reproducibilidad | No se puede volver a ejecutar | Se ejecuta con ayuda | Se ejecuta desde el README | Además incluye datos o script y versiones fijas |
| Análisis y comunicación al cliente | Solo código o cifras sin interpretar | Interpretación básica | Conclusiones claras para el cliente | Recomendación accionable con límites explicados |
| IA responsable | No se considera | Menciones genéricas | Riesgos específicos identificados | Medidas aplicadas y evaluadas |

*Pista de IA responsable:* ¿qué pasa si el país de origen se imputa mal? ¿Podría el modelo discriminar por nacionalidad? ¿Hay columnas que no deberían usarse?

**Pasos sugeridos.**

- **Miércoles 4/11:** *planning* (25 min) con el reparto de roles (Scrum Master y responsable técnico rotan); tras las prácticas 1–2, inventario de defectos con `resumen_calidad` y gráficos (60 min). Primer borrador de la tabla del informe.
- **Jueves 5/11:** función `limpiar_reservas`, comprobación antes/después y separación train/test; *pipeline* de preprocesado (práctica 3) y *baseline* con validación cruzada (práctica 4) (80 min).
- **Cierre del jueves:** informe, README y ensayo de 5 minutos (25 min); autoevaluación y coevaluación; **review + retrospectiva** (última hora).

**Guía paso a paso para principiantes**

***Paso 1 · Planning y reparto de roles (miércoles 4/11, 20 minutos)***

1. Elegid **Scrum Master y responsable técnico** de este sprint (no pueden repetir quienes lo fueron en el anterior).
2. Pasad al tablero las tareas: inventario de defectos, función de limpieza, pipeline, baseline, informe, README.
3. Decidid quién empieza cada una, sabiendo que **todo el equipo** debe entender todo.

***Paso 2 · Inventario de defectos (miércoles 4/11, 40 minutos)***

1. Cargad `reservas_sucio.csv` y ejecutad la función `resumen_calidad` de las prácticas. Mirad: tipos de columnas, nulos, duplicados, valores mínimos y máximos.
2. Haced gráficos (histogramas, conteo por categoría). A veces el defecto se ve mejor dibujado.
3. **Por cada defecto, rellenad una fila en la tabla del informe** (`informe_calidad_plantilla.md`):

| Campo | Ejemplo (inventado) |
|---|---|
| **Qué** | Fechas de llegada en dos formatos (`2025-07-03` y `03/07/2025`) |
| **Dónde** | Columna `fecha_llegada` |
| **Filas** | 183 de 2.000 |
| **Causa probable** | Dos sistemas de origen exportan con formato distinto |
| **Tratamiento** | Convertir todas a formato ISO con `pd.to_datetime(..., dayfirst=...)` |
| **Justificación** | No se pierde información y evita errores al ordenar por fecha |

Apuntad también las **preguntas que haríais al cliente** («¿qué significa un precio de 0 €? ¿es cortesía o un error?»).

***Paso 3 · Función de limpieza y separación train/test (jueves 5/11, 30 minutos)***

1. Escribid `limpiar_reservas(df)`: una función que recibe la tabla sucia y devuelve la tabla limpia aplicando **todos** vuestros tratamientos. **No toquéis el CSV a mano.**
2. **Comprobad antes y después** (nulos, duplicados, rangos). Si una corrección no se nota, no sirve.
3. **Separad train y test ANTES de calcular medianas, medias o categorías frecuentes.** Esos valores se aprenden **solo con train** y se aplican luego al test. Si calculáis la mediana con todos los datos, el test «se contamina» (fuga).
4. **No descartéis ninguna fila sin justificarlo** en el informe.

***Paso 4 · Pipeline y baseline (jueves 5/11, 30 minutos)***

1. Construid un `Pipeline` de scikit-learn con tres pasos: **imputar** huecos, **escalar** números y **codificar** categorías.
2. Añadid un **modelo de referencia** (por ejemplo, regresión logística).
3. Evaluadlo con **validación cruzada** y comparadlo con un `DummyClassifier`. Si vuestro modelo no mejora al «tonto», algo falla.
4. Pensad en la métrica: si solo se cancela el 20 %, un modelo que dice «nunca cancelan» acierta el 80 %. Por eso conviene mirar métricas que tengan en cuenta las cancelaciones (recall, F1…).

***Paso 5 · Informe, README y ensayo (jueves 5/11, 20 minutos)***

1. Completad el informe (1–2 páginas): tabla de defectos, tratamientos justificados y preguntas al cliente.
2. En `README.md`: cómo ejecutar el cuaderno, qué versiones de librerías usáis y en qué orden.
3. Comprobad que **«Reiniciar y ejecutar todo»** funciona sin errores.
4. Ensayad la review (5 minutos) y preparad la retro.

**Cómo usar la IA en este reto**

La limpieza de datos es un terreno donde la IA ayuda mucho, pero **donde también introduce errores sutiles**.

| Para qué | Ejemplo de petición | Qué tenéis que comprobar |
|---|---|---|
| Entender una función de pandas | «Explícame `fillna`, `drop_duplicates` y `to_datetime` con un ejemplo muy pequeño» | Que probáis cada ejemplo y lo entendéis |
| Depurar un error | «Este código da `KeyError: 'precio'`. El DataFrame tiene estas columnas: *[lista]*. ¿Qué puede pasar?» | Que arreglas la causa, no solo el síntoma |
| Revisar la fuga | «Aquí está mi pipeline: *[código]*. ¿Hay alguna operación que use el test para aprender?» | Que entendéis **por qué** hay o no fuga |
| Redactar el informe | «Reescribe esta tabla de defectos de forma más clara» | Que **no cambie cifras ni defectos** |

**Reglas del reto:**

- **Solo librerías del curso.** No instaléis paquetes extra que os «arreglen» los datos automáticamente.
- **Toda corrección debe estar en el código**, no en el CSV ni en un clic de Excel.
- Declarad en el README qué habéis hecho con IA y qué habéis verificado.
- **Pista de IA responsable:** ¿qué pasa si el país de origen se imputa mal? ¿Podría el modelo discriminar por nacionalidad? ¿Hay columnas que no deberían usarse? Respondedlo en el informe.

**Errores frecuentes (y cómo evitarlos)**

| Error | Por qué ocurre | Cómo evitarlo |
|---|---|---|
| Calcular la mediana con **todos** los datos antes de partir | «Total, es solo la mediana» | Partid primero; ajustad imputación y escalado **solo con train** (dentro del `Pipeline`) |
| Borrar filas «raras» sin avisar | Se quieren datos limpios | Cada fila eliminada se **cuenta y se justifica** |
| Informe que dice «hay nulos» sin más | Falta el análisis | Para cada defecto: dónde, cuántas filas, causa, tratamiento y razón |
| Un modelo que «acierta el 80 %» y se celebra | La exactitud engaña con clases desiguales | Comparadlo con `DummyClassifier` |
| Cuaderno que solo funciona si ejecutas las celdas en cierto orden | Se editaron celdas sueltas | *Reiniciar y ejecutar todo* |
| Usar columnas que delatan la respuesta | Parece que ayudan | Preguntad por cada una: ¿se conoce **al reservar**? |

**Un buen resultado se parece a...**

- **Tabla de defectos:** 6–10 filas completas con las seis columnas; los tres más graves destacados.
- **Función de limpieza:** una sola función, con docstring, que devuelve una tabla sin los defectos y **un resumen de lo que ha cambiado**.
- **Resultado del baseline:** una frase como *«Nuestro modelo obtiene un F1 medio de 0,58 ± 0,03, frente a 0,00 del modelo tonto»* (cifras de ejemplo).

**Antes de entregar: autoevaluación rápida**

| Pregunta | Sí / No |
|---|---|
| ¿El informe recoge todos los defectos con causa probable y tratamiento? | |
| ¿El cuaderno se ejecuta de principio a fin sin errores? | |
| ¿Imputación, escalado y codificación se ajustan **solo con train**? | |
| ¿Hay un baseline (`DummyClassifier`) y un modelo comparados con validación cruzada? | |
| ¿El README tiene instrucciones y versiones de librerías? | |
| ¿Hemos respondido a la pregunta de IA responsable (país de origen, discriminación)? | |
| ¿Está subido a Git, con autoevaluación y coevaluación? | |

**Si os atascáis**

- **No encontramos más defectos:** probad `df.describe()`, `df.isna().sum()`, `df.duplicated().sum()` y `df[columna].value_counts()` en cada columna.
- **No sabemos si algo es un error o un dato real:** apuntadlo como **pregunta para el cliente** y explicad qué hacéis mientras tanto.
- **Una función de pandas no hace lo que queremos:** probadla con un ejemplo de 3 filas inventadas.
- Más ayuda: Cómo se trabaja un reto.

**Qué se enseña en la review** (5 min por equipo): la tabla de defectos (los 3 más graves y por qué), el pipeline en marcha desde cero (`Restart & Run all`) y la métrica del baseline frente a un modelo "tonto".

**Preguntas de la retrospectiva:** ¿qué defecto nos costó más encontrar? ¿Qué decisión de limpieza discutimos más? ¿Qué haríamos distinto con el siguiente fichero del cliente?

## Definition of Done (lista de comprobación)

Si queda una casilla sin marcar, **el reto no está terminado**.

- ☐ El informe de calidad recoge todos los defectos encontrados con causa probable y tratamiento.
- ☐ El notebook se ejecuta de principio a fin sin errores (`Restart & Run all`).
- ☐ El pipeline ajusta imputación, escalado y codificación solo con train.
- ☐ Hay un baseline (`DummyClassifier`) y un modelo, comparados con validación cruzada.

- [ ] README con instrucciones y versiones de las librerías.
- [ ] Subido a Git; autoevaluación y coevaluación rellenadas.